

universidade de aveiro



deti

departamento de eletrónica,
telecomunicações e informática

Complements of Machine Learning

2025/2026



Complements of Machine Learning

Overview and Information

Course contents

- Advanced Computer Vision
 - CNN architectures, Object detection, Image segmentation, xAI
 - Transformers and Attention
 - Attention mechanisms, Transformer architecture
 - Vision Transformers and Image Generation
 - ViT, GAN, Diffusion models
 - Large Language Models
 - Architectures, Fine-tuning, Prompting, RAG
 - *Multi-Modal Learning*
 - *Deployment and ethical considerations*
- May not fit into 13 weeks!

Learning Outcomes

By course completion, you will:

- ✓ Implement modern neural architectures (CNNs, Transformers, ViTs)
- ✓ Fine-tune and deploy large language models
- ✓ Build RAG systems for domain-specific applications
- ✓ Apply explainable AI techniques
- ✓ Work with generative models (GANs, diffusion)
- ✓ *Design multi-modal AI systems*
- ✓ *Address ethical considerations in AI deployment*

Prerequisites Reminder

You should be comfortable with:

- ✓ Basic ML algorithms (linear/logistic regression, decision trees, SVMs)
- ✓ Simple neural networks and backpropagation
- ✓ Python programming
- ✓ Basic probability and linear algebra

We will review the fundamentals today!

Grading

Element	Weight	Due
In-class assignments	35%	1 week after class
Participation + notes	5%	Continuously
Project proposal	10%	31 March
Final project	50%	End of semester

Grading

- In-class assignments (35%)
 - Jupyter notebook (.ipynb) (+extra code if necessary)
 - All code cells executed with outputs
 - Markdown cells explaining approach
 - Results visualization
 - Brief reflection (what worked, what didn't)
 - Late Policy:
 - 10% deduction per day, up to 3 days
 - After 3 days: 50% maximum grade

- Weekly participation + notes (5%)
 - Participation during the class or through Discord (Q&A)
 - README.md updated weekly with (some or all of):
 - Notes taken during class
 - Things you learned / Questions you have
 - Comments and Suggestions

Assessment

Correctness	All tasks completed, results correct	Minor errors, mostly correct	Some major errors	Many errors or incomplete
Code Quality	Clean, documented, efficient	Readable, some documentation	Works but messy	Hard to follow
Analysis	Insightful interpretation of results	Good analysis	Basic observations	Minimal or missing
Presentation	Well-organized, clear visualizations	Organized, decent visuals	Somewhat disorganized	Confusing

Grading

- Project proposal (10%) (2-3 pages):
 - Problem Statement: What are you solving? Why is it interesting/important?
 - Dataset: What data will you use? Source, size, preprocessing needed. Show sample data.
 - Methodology: What models/techniques? Why these choices? Architecture diagram.
 - Evaluation Plan: What metrics? Baseline comparisons. Success criteria.
 - Ethics & Limitations: Potential biases. Limitations of approach. Responsible use considerations

Grading

- Final project (50%)
 - Code Repository (50%)
 - Clean, documented code; README with setup instructions; requirements.txt
 - Example usage; Reproducible results
 - Written Report (30%)
 - 6-8 pages (excluding references, appendices)
 - Conference paper format (template TBD)
 - Presentation (20%)
 - 10-12 minutes + 3 Q&A
 - All team members must present

Agenda for Today

Topic	Duration (mins)
Introduction & Course Overview	15
Theory Revision: Deep Learning Fundamentals	60
PyTorch Fundamentals	30
 Break	15
Hands-On Lab: Manual vs PyTorch Implementation	60

Learning Objectives for Today

1. Refresh core deep learning concepts
2. Gain practical PyTorch proficiency
3. Understand automatic differentiation
4. Understand NN layers in PyTorch

Deep Learning Fundamentals

1. Neural Network Architecture

From Perceptron to Deep Networks

From Perceptron to Multi-Layer Networks

The Perceptron (1957)

inputs $\rightarrow$ [weights] $\rightarrow \Sigma \rightarrow$
activation $\rightarrow$ output

$$y = f(\Sigma w_i x_i + b)$$

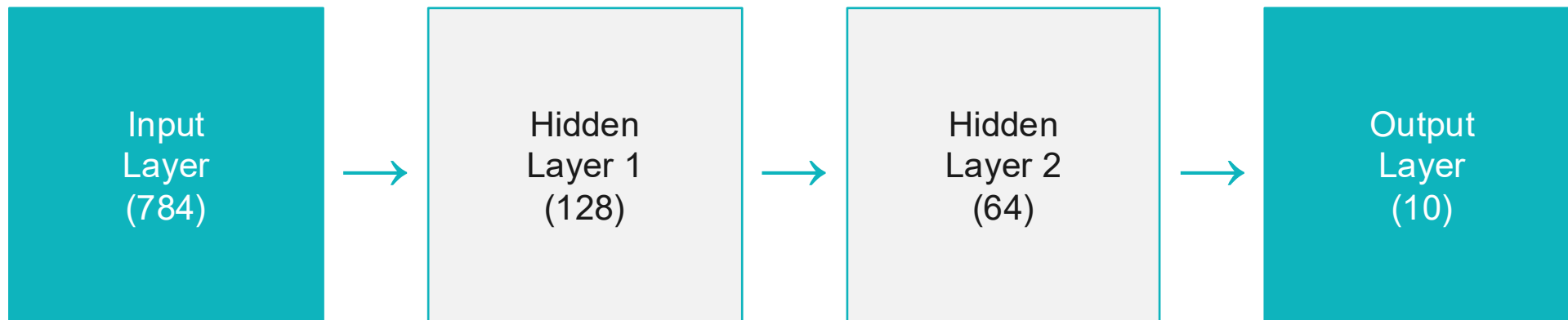
Limitation:

Can only learn linearly separable functions

Multi-Layer Perceptron (MLP)

- Stack multiple layers of neurons
- Each layer transforms the input
- Non-linear functions between layers enable learning complex patterns

Network Architecture



Key Components:

- Input Layer: Receives raw features
- Hidden Layers: Learn internal representations
- Output Layer: Produces predictions

Activation functions

Why Non-Linearity Matters!

Without activation functions:

$$f(f(f(x \cdot W_1) \cdot W_2) \cdot W_3) = x \cdot (W_1 \cdot W_2 \cdot W_3) = x \cdot W$$

Just a linear transformation! 

Common Activation Functions:

Function	Formula	Range	Use Case
Sigmoid	$\sigma(x) = 1/(1+e^{-x})$	$(0, 1)$	Binary output
Tanh	$\tanh(x)$	$(-1, 1)$	Zero-centered
ReLU	$\max(0, x)$	$[0, \infty)$	Hidden layers (default)
Leaky ReLU	$\max(\alpha x, x)$	$(-\infty, \infty)$	Avoids dead neurons

Modern Default: ReLU for hidden layers (simple, efficient, works well)

Universal Approximation Theorem

Conceptual Understanding

A neural network with a single hidden layer can approximate any continuous function on a compact subset of $\mathbb{R}^n$, given an appropriate activation function.

In Practice

- Deep networks (multiple layers) are more efficient
- Fewer parameters needed for same expressive power
- Better generalization in practice

2. Forward and Backward Propagation

How Neural Networks Learn

Forward Pass: Layer by Layer

Matrix Operations (for a single layer):

$$z^{(1)} = W^{(1)} \cdot a^{(1-1)} + b^{(1)}$$

$$a^{(1)} = g(z^{(1)})$$

Example: 2-Layer Network

```
# Layer 1: Input (784) → Hidden (128)
z1 = W1 · x + b1      # Linear
a1 = relu(z1)         # Activation

# Layer 2: Hidden (128) → Output (10)
z2 = W2 · a1 + b2
output = softmax(z2)
```

Loss Functions

Regression: Mean Squared Error (MSE)

$$L_{\text{MSE}} = (1/n) \sum (y_i - \hat{y}_i)^2$$

Classification: Cross-Entropy Loss

$$L_{\text{CE}} = -\sum y_i \log(\hat{y}_i)$$

Binary:

$$L_{\text{BCE}} = -[y \log(\hat{y}) + (1-y) \log(1-\hat{y})]$$

Key Insight: Loss function measures how wrong we are → goal is to minimize it

Gradient Descent: Parameter Updates

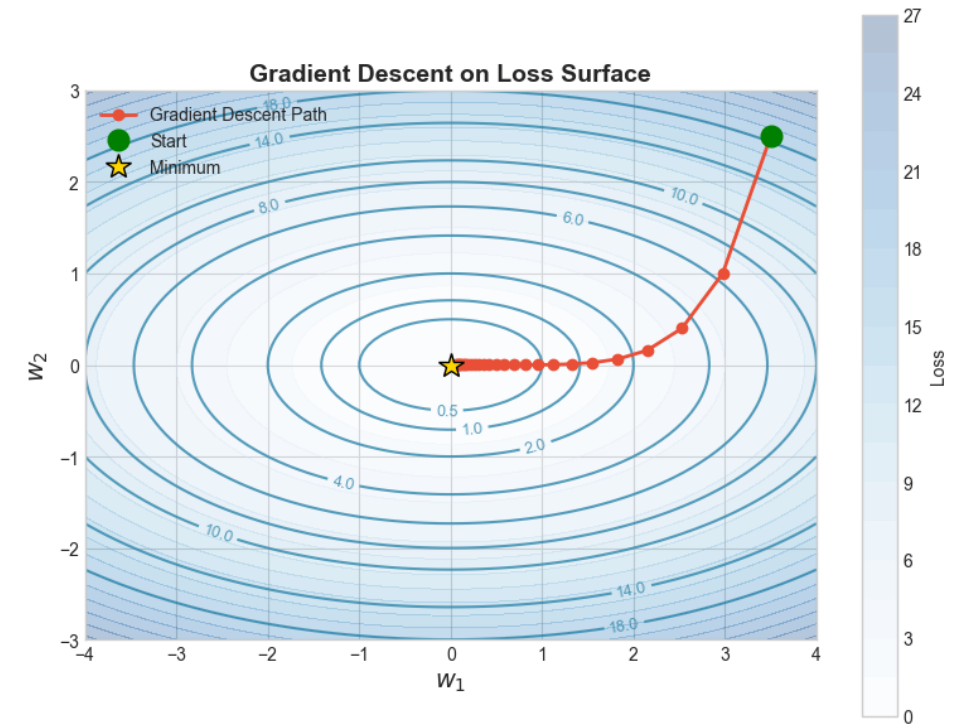
Update Rule:

$$W^{(1)} = W^{(1)} - \alpha \cdot \partial L / \partial W^{(1)}$$

Where α is the learning rate

How it works:

- Start at current weights (high loss)
- Move in gradient direction (steepest descent)
- Arrive at updated weights (lower loss)



Backpropagation: Chain Rule Intuition

The Problem:

We need $\partial L / \partial W^{(l)}$ for every layer !

The Solution: Chain Rule

$$\partial L / \partial W^{(1)} = \partial L / \partial a^{(1)} \cdot \partial a^{(1)} / \partial z^{(1)} \cdot \dots \cdot \partial z^{(1)} / \partial W^{(1)}$$

Intuition:

Input $\rightarrow$ Hidden_1 $\rightarrow$... $\rightarrow$ Hidden_{L-1} $\rightarrow$ Hidden_L $\rightarrow$ Output $\rightarrow$ Loss

$\leftarrow \leftarrow \leftarrow \leftarrow$ Gradients flow backward $\leftarrow \leftarrow \leftarrow \leftarrow$

We propagate error signals backward through the network

3. Optimization and Training

Making Networks Learn Efficiently

Gradient Descent Variants

Batch Gradient Descent

✓ Stable convergence

✗ Slow, memory intensive

Stochastic GD (SGD)

✓ Fast updates,
escapes local minima

✗ Noisy, unstable

Mini-Batch GD ✓

✓ Best of both worlds
Typical: 32, 64, 128, 256

Modern Optimizers

SGD with Momentum:

$$v_t = \beta \cdot v_{t-1} + \nabla L$$

$$W = W - \alpha \cdot v_t$$

↳ Overcomes local minima and saddle points

Adam (Adaptive Moment Estimation) ✓

$$m_t = \beta_1 \cdot m_{t-1} + (1 - \beta_1) \cdot \nabla L$$

$$v_t = \beta_2 \cdot v_{t-1} + (1 - \beta_2) \cdot (\nabla L)^2$$

$$W = W - \alpha \cdot m_t / (\sqrt{v_t} + \epsilon)$$

Combines momentum + adaptive learning rates

Most popular optimizer!

Learning Rate Impact

High LR (0.1): Diverges or oscillates wildly

Good LR (0.01): Converges well ✓

Low LR (0.0001): Converges but very slowly

Learning Rate Scheduling:

- Step decay: Reduce LR at specific epochs
- Exponential decay: $\alpha_t = \alpha_0 \cdot e^{-kt}$
- Cosine annealing: Smooth periodic reduction
- Warmup: Start small, increase, then decay

Batch Size Considerations

Size	Pros	Cons
Small (16-32)	Regularization effect, less memory	Noisy, slower
Medium (64-256)	Good balance	-
Large (512+)	Faster training, stable gradients	Need LR scaling, worse generalization

Rule of Thumb:

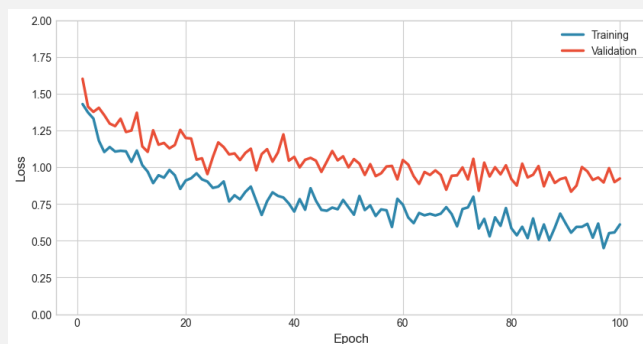
- Start with 32 or 64
- Increase if training is slow and memory allows
- If using large batch, scale learning rate (linear scaling rule)

4. Regularization and Generalization

Preventing Overfitting

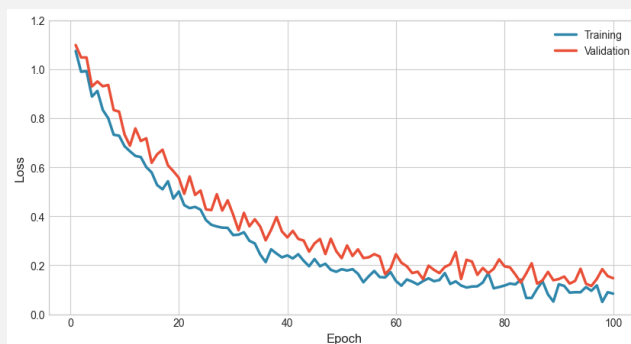
Overfitting vs Underfitting

Underfitting



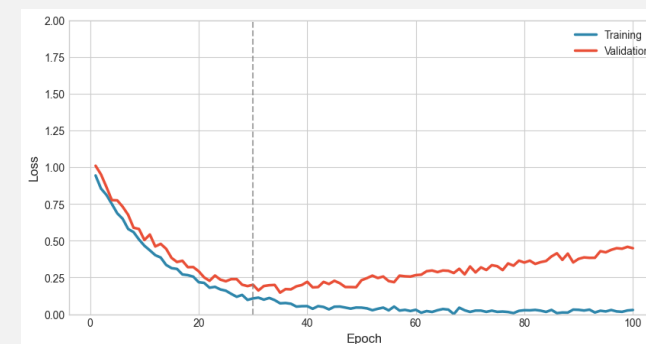
Train and val losses high
Model is too simple

Good Fit ✓



Val follows train
Good generalization

Overfitting



Val diverges from train
Memorizing training data

Regularization Techniques: L1 & L2

L1 Regularization (Lasso):

$$L_{\text{total}} = L_{\text{data}} + \lambda \sum |w_i|$$

→ Promotes sparsity (many weights → 0)

L2 Regularization (Ridge/Weight Decay):

$$L_{\text{total}} = L_{\text{data}} + \lambda \sum w_i^2$$

→ Encourages small weights, prevents extremes

Typical λ values: $1e-4$ to $1e-2$

Dropout

Training Mode:

Randomly set neurons to 0 with probability p

Layer Input: [0.5, 0.3, 0.8, 0.2, 0.6]

Dropout Mask: [1, 0, 1, 0, 1]

Output: [0.5, 0, 0.8, 0, 0.6] / (1- p)

Why It Works:

- Prevents co-adaptation of neurons
- Ensemble effect: training many subnetworks
- Typical values: $p = 0.2$ to 0.5

Inference Mode:
No dropout applied
(or scale by 1- p)

Batch Normalization

The Idea:

Normalize activations within each mini-batch

$$x' = (x - \mu_{\text{batch}}) / \sqrt{(\sigma^2_{\text{batch}} + \epsilon)}$$
$$y = \gamma \cdot x' + \beta$$

Benefits:

- ✓ Faster training (allows higher learning rates)
- ✓ Reduces internal covariate shift
- ✓ Slight regularization effect
- ✓ Less sensitive to initialization

Placement:
Usually before
activation function

Train / Validation / Test Split

Training Set (60-80%)

Learn parameters
(weights, biases)

Validation Set (10-20%)

Tune
hyperparameters

Test Set (10-20%)

Final evaluation
(NEVER touch!)

Common Mistakes

- Using test set to select hyperparameters
- Data leakage between sets
- Not shuffling before splitting

PyTorch Fundamentals

Tensors: The Foundation

```
import torch

# From Python lists
x = torch.tensor([1.0, 2.0, 3.0])

# Zeros, ones, random
zeros = torch.zeros(3, 4)
ones = torch.ones(2, 3, 4)
random = torch.randn(5, 5) # Normal distribution

# From NumPy
tensor = torch.from_numpy(np.array([1, 2, 3]))
```

Tensor Operations

```
# Arithmetic
c = a + b          # Element-wise addition
d = a * b          # Element-wise multiplication
e = a @ b          # Dot product
f = torch.matmul(A, B) # Matrix multiplication

# Reshaping
y = x.view(16)      # Flatten
z = x.view(2, 8)    # Reshape
w = x.reshape(2, -1) # -1 infers dimension

# Moving to GPU
x = x.to('cuda')
```

Autograd: Automatic Differentiation

```
# Create tensor with gradient tracking
x = torch.tensor([2.0, 3.0], requires_grad=True)

# Forward pass
y = x ** 2
z = y.sum()

# Backward pass - compute gradients
z.backward()

print(x.grad)  # tensor([4., 6.]) = 2*x
```

Key Methods:

- `.backward()`
Compute gradients
- `.grad`
Access gradients
- `.detach()`
Remove from graph
- `torch.no_grad()`
Disable tracking

nn.Module: The Base Class

```
import torch.nn as nn

class MyNetwork(nn.Module):
    def __init__(self):
        super().__init__()
        self.layer1 = nn.Linear(784, 128)
        self.layer2 = nn.Linear(128, 10)

    def forward(self, x):
        x = torch.relu(self.layer1(x))
        return self.layer2(x)

model = MyNetwork()
output = model(input_tensor)  # Calls forward()
```

Common Layers

Linear (Fully Connected):

```
nn.Linear(in_features, out_features)
```

Activation Functions:

```
nn.ReLU(), nn.Sigmoid(), nn.Tanh()  
nn.LeakyReLU(negative_slope=0.01)
```

Regularization:

```
nn.Dropout(p=0.5)  
nn.BatchNorm1d(num_features)
```

For Images (Week 2):

```
nn.Conv2d(in_ch, out_ch, kernel)  
nn.MaxPool2d(kernel_size)
```

Loss Functions & Optimizers

Loss Functions:

```
nn.MSELoss()           # Regression
nn.CrossEntropyLoss()  # Multi-class
nn.BCEWithLogitsLoss() # Binary
```

Optimizers:

```
optim.SGD(params, lr=0.01)
optim.Adam(params, lr=0.001) # ✓
optim.AdamW(params, lr=0.001)
```

 *Don't apply softmax before CrossEntropyLoss!*

The Training Loop Pattern

```
model = MyNetwork()
criterion = nn.CrossEntropyLoss()
optimizer = optim.Adam(model.parameters(), lr=0.001)

for epoch in range(num_epochs):
    for batch_x, batch_y in train_loader:

        optimizer.zero_grad()      # 1. Zero gradients

        outputs = model(batch_x) # 2. Forward pass
        loss = criterion(outputs, batch_y)

        loss.backward()            # 3. Backward pass

        optimizer.step()           # 4. Update weights
```



15 minutes

Part C: Hands-On Lab

60 minutes

Lab Overview

Goal: Build a 2-layer neural network for MNIST digit classification

Two Implementations:

1. Manual Implementation (30-35 min)

Implement layers from scratch

2. PyTorch Implementation (20-25 min)

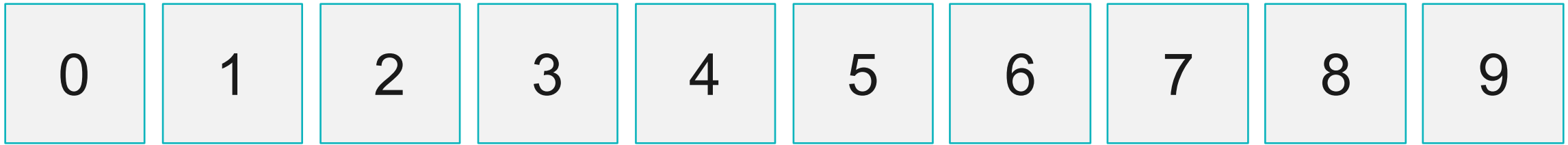
Use nn.Module and built-in layers

3. Comparison & Extensions (remaining time)

Compare and add regularization

MNIST Dataset

28×28 grayscale images of handwritten digits



60,000 training images + 10,000 test images

Input: 784 features (28×28 flattened)

Output: 10 classes (digits 0-9)

Task 1: Manual Implementation

```
class ManualLayer:
    def __init__(self, input_dim, output_dim):
        self.W = torch.randn(input_dim, output_dim,
                               requires_grad=True) * 0.01
        self.b = torch.zeros(output_dim, requires_grad=True)

    def forward(self, x):
        # TODO: Implement output = x @ W + b
        pass

class ManualReLU:
    def forward(self, x):
        # TODO: Implement max(0, x)
        pass
```

Task 2: PyTorch Implementation

```
class PyTorchNN(nn.Module):
    def __init__(self):
        super().__init__()
        self.layer1 = nn.Linear(784, 128)
        self.relu = nn.ReLU()
        self.layer2 = nn.Linear(128, 10)

    def forward(self, x):
        # TODO: x -> layer1 -> relu -> layer2 -> output
        pass

model = PyTorchNN()
criterion = nn.CrossEntropyLoss()
optimizer = optim.Adam(model.parameters(), lr=0.001)
```

Task 3: Extensions to Try

1. Compare Optimizers

Train with SGD vs Adam, plot loss curves

2. Vary Learning Rate

Try 0.1, 0.01, 0.001, 0.0001

3. Add More Layers

What happens with 3, 4, 5 layers?

4. Add Regularization

Add Dropout and BatchNorm

Common Issues & Solutions

Issue	Solution
Shape mismatch error	Print shapes: <code>print(x.shape)</code>
Gradients are None	Check <code>requires_grad=True</code> is set
Model not learning	Check learning rate, verify labels
CUDA/CPU device error	Ensure data and model on same device
NaN in loss	Learning rate too high, numerical instability

Deliverable

Submit: Working Jupyter Notebook with:

- ✓ Manual implementation (completed code)
- ✓ PyTorch implementation (completed code)
- ✓ Training results for both
- ✓ Comparison analysis (brief written discussion)
- ✓ At least one extension experiment

Due: Next week before class

Summary

What We Covered Today

What We Covered Today

Theory:

- Neural network architecture (perceptron → MLP)
- Forward and backward propagation
- Loss functions (MSE, Cross-Entropy)
- Optimization (SGD, Adam)
- Regularization (Dropout, BatchNorm, L1/L2)

PyTorch:

- Tensors and operations
- Autograd (requires_grad, backward())
- nn.Module structure
- Training loop pattern

Key Takeaways

1. **Non-linearity is essential** — without it, deep = shallow
2. **Backpropagation is just the chain rule** — PyTorch handles it
3. **Adam is a good default optimizer** — but SGD+momentum works too
4. **Regularization prevents overfitting** — use dropout, batch norm
5. **Understanding manual implementation helps debugging**

Looking Ahead

Next Week: Modern CNN Architectures

- ResNet and skip connections
- EfficientNet and compound scaling
- Transfer learning with pre-trained models

Preparation:

- Read: ResNet paper (He et al., 2015) - sections 1-3
- Review: CNN basics from Semester 1

Resources

Documentation:

- pytorch.org/tutorials/
- pytorch.org/docs/

Videos:

- 3Blue1Brown: Neural Networks series
- Andrej Karpathy: Neural Networks: Zero to Hero

Books:

- "Deep Learning" by Goodfellow et al.
- "Dive into Deep Learning" (free online)

GPU Access:

- Google Colab (free tier sufficient)
- Kaggle Notebooks

Thank You!

See you next week!
